

An Implementation of Spin, MCS, and CLH Locks

Omar Attaalla

April 2025

1 Introduction

Race conditions have been a significant problem in Computing resulting in decades of research and multiple solutions. The problem revolves around ensuring a method's critical section is atomic. That is, it runs entirely or not at all. C++ supports atomic operations. However, this does not entirely solve the problem when multiple threads can access the same memory. In this scenario, a method that only allows one thread to execute important instructions (critical section) is needed. This problem has led to the creation of Locks which allow threads to acquire an exclusive resource that is required to execute their critical section. In particular, this paper will focus on the implementation of three popular locking mechanisms: Test and Test and Set (TTAS) Spin Locks, MCS Locks, and CLH Locks. This implementation is written in C++ using the atomic standard library.

2 Spin Locks

2.1 About Spin Locks

Spin Locks are a standard lock that use the idea of conditional waiting (busy-waiting) to avoid entering its critical section if another thread is in it's critical section. Since this lock simply waits in a loop, it uses unnecessary computational resources. However, Spin Locks are much simpler to implement and has less overhead than the two other locks we will discuss.

2.2 Test and Test and Set Spin Lock Implementation

Any lock's design involves it's initialization, locking, and unlocking methods. We first introduce an abstract class Lock which has abstract methods lock() and unlock(). This allows us to simplify our test bench implementation(discussed later):

```
class Lock {
public:
    virtual void lock() = 0;
    virtual void unlock() = 0;
};
```

We then create a SpinLock class which initializes an atomic bool as false.

```
class SpinLock : public Lock {
private:
    std::atomic<bool> state = false;
    .
    .
}
```

2.2.1 Spin Lock Locking Implementation

The lock() method involves a continuous conditional wait as shown below:

```
void lock() {
    while (true) {
        while (state.load(std::memory_order_relaxed)) {}
        if (!state.exchange(true, std::memory_order_acquire)) {
            return;
        }
    }
}
```

We start by constantly fetching the current state from memory using memory order relaxed, which ensures atomicity without performance degradation. However, this fetch does not guarantee any ordering (which does not matter for this read). We then perform an exchange which replaces the state with true (stating it is locked) while returning the old value. If the old value is false then we have acquired the lock and can simply return. However, we must use memory order acquire to indicate to the other threads that we have acquired the lock.

2.2.2 Spin Lock Unlocking Implementation

The unlock method simply performs a release by setting the state to false (unlocked):

```
void unlock() {
    state.store(false, std::memory_order_release);
}
```

3 CLH Locks

3.1 About CLH Locks

CLH Locks are part of the queue based family of locks that maintain a list of waiting threads. This type of lock is fair since ordering is guaranteed. That is, lock acquisitions are made in a FIFO order.

3.2 CLH Locks Implementation

Since CLH Locks use a linked list style of queue, we first define a cNode which represents a node in our lock queue:

```
class cNode {
public:
    std::atomic<bool> locked;
    cNode() : locked(false) {}
    cNode(bool state) : locked(state) {}
};
```

This node maintains the current state "locked" which defines if the lock is held. We then define a CLHLock class which represents the CLH lock:

```
class CLHLock : public Lock {
private:
    std::atomic<cNode*> tail;
    thread_local static cNode* node;
```

```

    thread_local static cNode* prev;

public:
    CLHLock() {
        tail.store(new cNode(false), std::memory_order_relaxed);
    }

```

This class holds a tail which is a global atomic pointer to the last node that is waiting for the lock. It also maintains two thread local nodes that represent the node of the thread and a copy of the threads predecessor. These two nodes are initialized using the C++ `thread_local` keyword.

3.2.1 CLH Locks Locking Implementation

The locking mechanism of the CLH Lock differs from the spinlock in that we utilize the queue structure we have built. However, since the CLH lock is a spin based lock, we still loop until the lock is available:

```

void lock() {
    node->locked.store(true, std::memory_order_relaxed);
    prev = tail.exchange(node, std::memory_order_acq_rel);
    while (prev->locked.load(std::memory_order_acquire)) {}
}

```

We first set the locked attribute of our thread local node to true to indicate that we are attempting to acquire the lock. We then swap with the current predecessor in the queue if they exist. This exchange requires both acquire and release memory ordering to guarantee acquire atomicity on read and release atomicity on write. This exchange also returns the previous predecessor that the current thread is replacing. We then simply wait until the previous predecessor releases the lock (using memory order acquire since we are attempting to acquire the shared value).

3.2.2 CLH Locks Unlocking Implementation

The unlock method of the CLH Lock is similar to a normal spin lock where the state is set to false with a release memory order (notifying the successor):

```

void unlock() {
    node->locked.store(false, std::memory_order_release);
    node = prev;
}

```

When an unlock is performed, the successor re-uses it's predecessor's previous node (as denoted by `node = prev`).

4 MCS Locks

4.1 About MCS Locks

MCS Locks differ from CLH Locks in that it spins on it's own nodes value instead of spinning on the predecessor nodes value. This benefits systems that implement a NUMA Architecture since the predecessors node is often remote and expensive to access. The full implementation is included in the source code.

5 Test-bench

In order to measure the correctness and performance of the implementation of CLH, MCS, and Spin Locks, we have created a test bench that makes use of the standard C++ thread library. This Test-bench includes the creation of multiple threads that compete for a lock.

5.1 Test-bench Design and Implementation

The test-bench creates multiple threads that run either a simple counter or a computationally extensive critical section. We decided to measure both a simple counter function and a time-intensive critical section to measure how the size of the critical section impacts each of the different locking mechanisms.

```
void testMain(Lock* lock, bool heavy = false) {
    double avgClocks[NUM_THREADS] = {0};

    for (int i = 0; i < NUM_ITERATIONS_AVG; ++i) {
        for (int i = 1; i <= NUM_THREADS; ++i) {
            double avgAcquireTime = 0;
            if (heavy) {
                avgAcquireTime = testLock(lock, i, true);
            } else {
                avgAcquireTime = testLock(lock, i);
            }
            avgClocks[i] += avgAcquireTime;
        }
    }
}
```

In each test, we run the main function multiple times (NUM_ITERATIONS_AVG times) in order to get an average time each thread spent acquiring a single lock. The testLock() method spawns i threads that call a method that acquires one of the locks. We can then ensure that our lock implementation is correct by asserting if the final value of the counter is the value we expect:

```
assert(counter == NUM_ITERATIONS_PER_THREAD * nthreads);
```

Each thread runs the lock acquisition process NUM_ITERATIONS_PER_THREAD times. In our testing we set NUM_ITERATIONS_PER_THREAD to 100,000 which means that each thread attempts to acquire the lock 100,000 times and increments the counter.

```
for (int i = 0; i < NUM_ITERATIONS_PER_THREAD; ++i) {
    clock_t start = clock();
    lock->lock();
    clock_t end = clock();
    (*counter)++;
    lock->unlock();
}
```

The testLock() method maintains a vector of threads, a counter, and an atomic acquireTime variable to measure the average time each thread spends acquiring a single lock.

```
double testLock(Lock* lock, int nthreads, bool heavyWork = false) {
    int counter = 0;

    std::vector<std::thread> threads;
    std::atomic<int> acquireTime(0);

    for (int i = 0; i < nthreads; ++i) {
        if (heavyWork) {
            threads.emplace_back(heavyIncr, lock, &counter, &acquireTime);
        } else {
            threads.emplace_back(incr, lock, &counter, &acquireTime);
        }
    }
}
```

```

    }
}
.
.

```

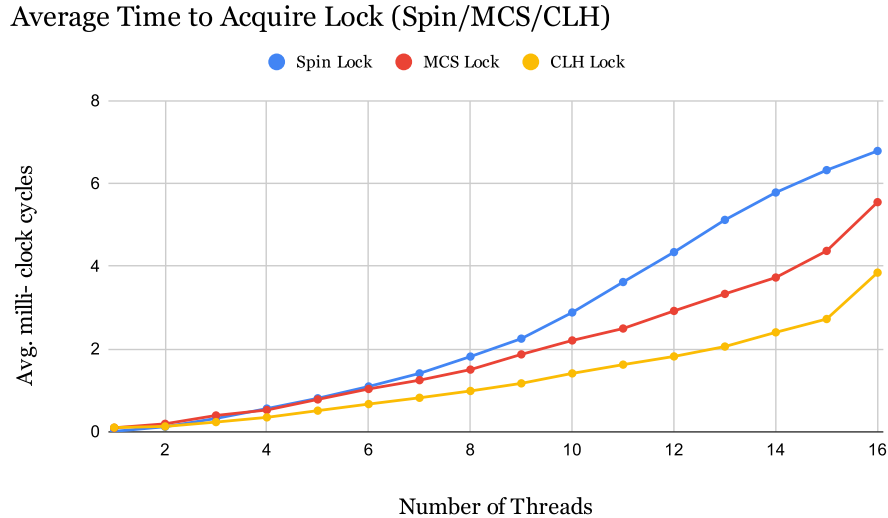
The implementation above shows how the threads are created with either a `heavyIncr()` function (simulates a large critical section) or a simple `incr()` function. Note that our testing was constrained by the number of cpu threads we had access to. Our system could only access up to 16 threads.

5.2 Test-bench Results

The main performance metric we measured was the average time each thread spent waiting for a lock. This is important for our system since if the thread is waiting for a lock, it is wasting computational resources by spinning instead of doing useful work. Each thread marked the time it started waiting for a lock and the time it acquired that lock. We tested this with 1 to 16 threads (each thread running 100,000 lock acquisitions).

5.2.1 Simple Counter Results

The results of using a simple counter as the critical section are shown in the figure below:



Each thread measured the amount of clock cycles that have passed for it to acquire a single lock. Note that some values show a clock cycle less than 1. This is because `ctime's clock()` method approximates the number of clock cycles that have passed and returns 0 for a good number of the acquisitions. However, this does not influence the results since we are focused on the trend on the number of clock cycles rather than the number of clock cycles itself.

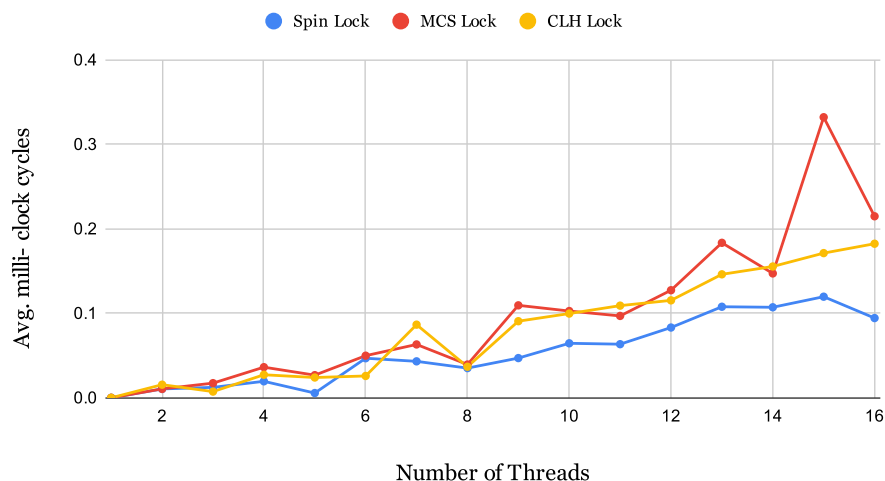
As expected we saw the Spin Lock perform the worst, in that clock acquisitions were the slowest. Although spin locks have little overhead, the spinning mechanism that requires many expensive atomic instructions causes it to perform poorly and fail to scale when we have high contention (more than 7 threads).

We also saw that the CLH Lock performed best. This is likely because the CLH Lock has less overhead in its implementation compared to the MCS Lock. Additionally, our test-bench was run on a non-NUMA architecture system that does not benefit from the local cache work performed by an MCS Lock. However, both the MCS Lock and CLH scaled similarly.

5.2.2 Heavy Work Results

In this section, we simulated a large critical section by running many iterations of multiplications. This measures how a larger critical section impacts each locking mechanism's performance. Since we are limited by compute, each thread ran 100 iterations instead of 100,000 iterations. In each critical section, 10,000 multiplications were performed. The results of running this test are shown below:

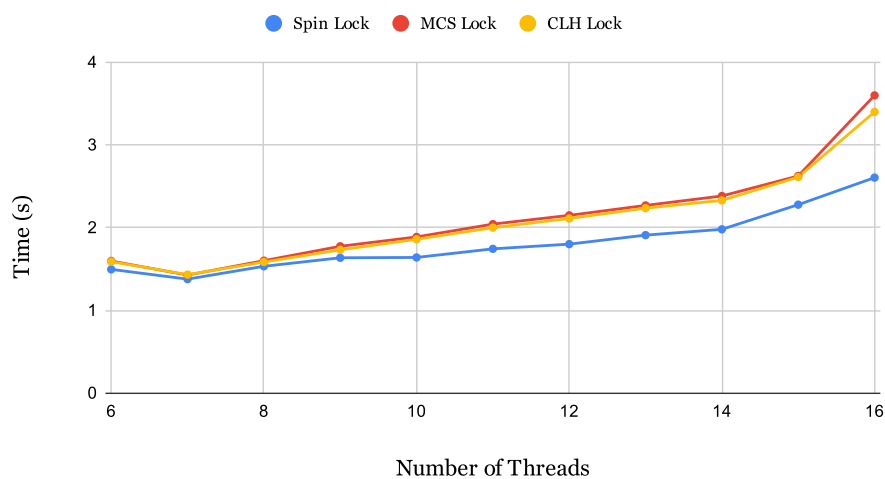
Average Time to Acquire Lock with Intensive Critical Section (Spin/MCS/CLH)



Against expectations, the average time to acquire a lock was the least when using the TTAS Spin Lock. This is likely because the overhead of the MCS/CLH Locks outweighed the constant aggressive spinning of the spin lock. The CLH and MCS Lock performed similarly.

To measure the overall performance gained by multi threading when using different locking mechanics, we again surprisingly saw the TTAS Spin Lock perform best in the figure below:

Computation Completion Time vs. Number of Threads



The overall time spent on computation was reduced from about 38s to about 1.4s when using multi threading. However, once we reached 7 threads, the lock contention was high enough that we saw performance degrade. The overhead of each thread waiting for a lock saw the performance get worse after about 7 threads.

6 Conclusion

In this paper, we implemented a Test and Test and Set Spin Lock, an MCS Lock, and a CLH Lock. We then measured the performance of each lock by measuring the average time each thread spent waiting for a lock. We also measured the overall performance gained by splitting the work across multiple threads using the different locking mechanisms. Overall, the CLH lock performed best in that threads spent the least time waiting for acquire this type of lock. However, when measuring the overall performance of the system (and when heavy critical section work was involved) we surprisingly saw that the TTAS lock performed best. We believe that the TTAS lock performed best because of its limited overhead (overhead required to implement queues in CLH/MCS Locks). We also believe our testing system led to the poor performance of CLH/MCS Locks since we were limited by a 16 thread non-NUMA architecture machine.